

Machine Learning Exercise 12

EXERCISE 1

In the lecture we have defined two string kernels K_D in the form of a convolution kernel (String Kernels 1 and 2). We found that Kernel 2 is the same as the n -spectrum kernel we have previously encountered (Lecture 3, slide 23). What is the equivalent definition of convolution Kernel 1 in terms of a feature vector (similar to the features ϕ_u, ϕ_u^+ we have defined in lecture 3)?

Solution:

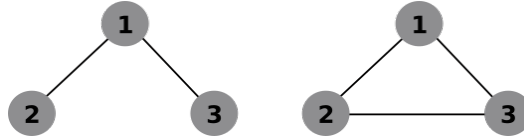
Similar as for the n -spectrum kernel: for every $u \in \Sigma^n$ define for a string $s \in \Sigma^*$ the feature

$$\phi_u^{0/1}(s) = \begin{cases} 1 & \text{if } u \text{ appears as a substring in } s \\ 0 & \text{otherwise} \end{cases}$$

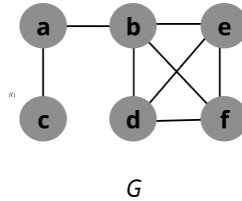
This defines for $s \in \Sigma^*$ the feature vector $\phi^{0/1}(s) = \{\phi_u^{0/1}(s) | u \in \Sigma^n\}$. The dot product $\phi^{0/1}(s) \cdot \phi^{0/1}(s')$ for two strings s, s' then just returns the number of vector components that are equal to 1 in both $\phi^{0/1}(s)$ and $\phi^{0/1}(s')$, i.e., the number of u appearing in both s and s' .

EXERCISE 2

For the two graphlets g_1, g_2 of size 3:



and the graph G :



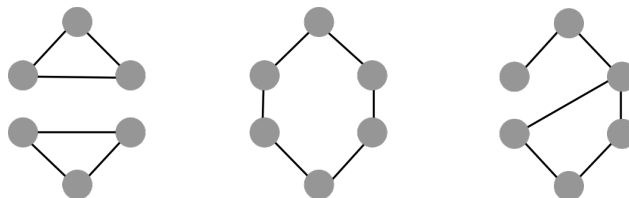
compute the feature values $\phi_{g_1}^{(1)}(G), \phi_{g_2}^{(1)}(G), \phi_{g_1}^{(2)}(G), \phi_{g_2}^{(2)}(G)$

Solution:

$$\begin{aligned} \phi_{g_1}^{(1)}(G) &= 8 \\ \phi_{g_2}^{(1)}(G) &= 24 \\ \phi_{g_1}^{(2)}(G) &= 4 \\ \phi_{g_2}^{(2)}(G) &= 4 \end{aligned}$$

EXERCISE 3

Recall the three graphs from slide 22:



A. Perform the first three rounds of WL code refinement:

- Start with each node having assigned the same code value
- Update (three times) according to the rule on slide 21. You can freely choose how you want to represent code values c^k (the code format on the slide is just an example), but you have to be sure that codes are unique in that the *hash* function returns different code values for different inputs.

B. Construct another graph for which the code values of all nodes will be the same in each iteration as for the nodes in the first two graphs above.

Solution:

(Sketch). We assign all nodes (in all graphs) the initial code '1' (in quotes to indicate that we see this as a string, rather than an integer value). Every node v in the first two graphs then has two neighbors with code '1', and has itself code '1' also. Thus

$$c^1(v) = \text{hash}('1', \{\text{'1'}, \text{'1'}\})$$

Lets give this value of $c^1(v)$ the code value '01'. The same repeats in the next two iterations: all nodes have two neighbors with identical code values, and also have the same code value themselves, to the next $c^k(v)$ will be always the same.

Now consider the third graph. Label the nodes a,b,c,d,e,f in clockwise order starting from the top node. Then

$$\begin{aligned} c^1(a) = c^1(c) = c^1(d) = c^1(e) &= \text{hash}('1', \{\text{'1'}, \text{'1'}\}) \\ c^1(b) &= \text{hash}('1', \{\text{'1'}, \text{'1'}, \text{'1'}, \text{'1'}\}) \\ c^1(f) &= \text{hash}('1', \{\text{'1'}\}) \end{aligned}$$

Call these three code values '01', '02', '03'. Continue in the same manner for two more rounds. In the second round, nodes a,c,d will receive different codes. Nodes c and e will still have the same, and that stays this way for the following iteration.